Postfix Algorithms

Algorithm Evaluate (P)
Evaluate a postfix expression P

```
create an empty stack S;
index ← 1;
while we have not reached the end of P do
   ch ← P[index]; {store in ch the next character in P}
   if ch is an operand then
      push ch onto S;
   else {ch is an operator}
      operand1 ← pop from S;
      operand2 ← pop from S;
      answer ← operation ch performed on operand1 and operand2;
      push answer onto S;
   end if
   index ← index + 1;
end while
result ← pop from S;
return result
```

Algorithm InfixToPostFix (I)
Transform an infix expression I to a postfix expression P

```
create an empty stack S;
P ← empty expression;
index ← 1;
while we have not reached the end of I do
   ch ← I[index]; {store in ch the next character in I}
   if ch is an operand then
      append ch to the end of P;
   else if ch is a '(' then
      push ch onto S;
   else if ch is a ')' then
      repeat
         pop operators from S and append them to P;
      until a '(' is popped;
   else {ch is an operator}
      while S is not empty and top of S is not '(' and top of S is not a lower precedence operator do
         pop operators from S and append them to P;
      end while
      push ch onto S;
   end if
   index ← index + 1;
end while
```